

UNIT 8: TREES, SPANNING TREES, AND GRAPH TRAVERSAL

Structure, Algorithms, Search Strategies, and Optimization

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



Course Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary



Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary



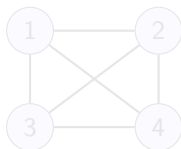
Spanning Tree Definition

Definition

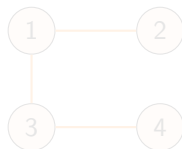
A **spanning tree** of a connected graph $G = (V, E)$ is a subgraph $T = (V, E')$ where:

- T is a tree (connected and acyclic)
- T includes all vertices of G (spanning property)
- $|E'| = |V| - 1$ edges

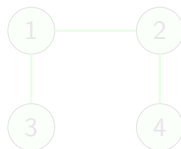
Original Graph (6 edges)



Spanning Tree 2 (3 edges)



Spanning Tree 1 (3 edges)



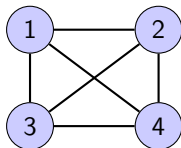
Spanning Tree Definition

Definition

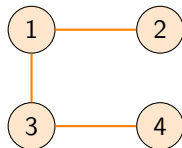
A **spanning tree** of a connected graph $G = (V, E)$ is a subgraph $T = (V, E')$ where:

- T is a tree (connected and acyclic)
- T includes all vertices of G (spanning property)
- $|E'| = |V| - 1$ edges

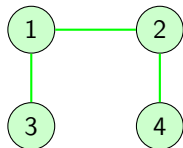
Original Graph (6 edges)



Spanning Tree 2 (3 edges)



Spanning Tree 1 (3 edges)



Properties of Spanning Trees I

Structural Properties

- **Minimality:** Spanning tree with fewest edges that maintains connectivity
- **Uniqueness:** Not unique for most graphs (multiple spanning trees possible)
- **Edge count:** Always $|V| - 1$ edges
- **Connectivity:** Unique path between any two vertices



Properties of Spanning Trees II

Fundamental Property

Any set of $|V| - 1$ edges that forms a tree in G is a spanning tree if and only if all vertices are included.

Number of Spanning Trees

Computed using Kirchhoff's Matrix Tree Theorem. For K_n : n^{n-2} distinct spanning trees.

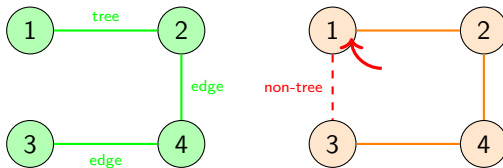


Fundamental Circuits

Fundamental Circuit

Adding a **non-tree edge** (edge not in spanning tree T) to T creates exactly one cycle, called a **fundamental circuit**.

Spanning Tree (T) Add Non-Tree Edge: Cycle



Key Property

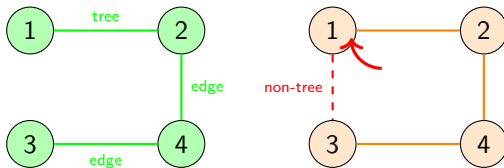
Each non-tree edge creates **exactly one** fundamental circuit. The set of all fundamental circuits forms a basis for the cycle space of G .

Fundamental Circuits

Fundamental Circuit

Adding a **non-tree edge** (edge not in spanning tree T) to T creates exactly one cycle, called a **fundamental circuit**.

Spanning Tree (T) Add Non-Tree Edge: Cycle



Key Property

Each non-tree edge creates **exactly one** fundamental circuit. The set of all fundamental circuits forms a basis for the cycle space of G .

Rank and Nullity

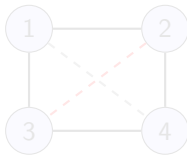
Rank and Nullity

For a graph $G = (V, E)$:

- **Rank:** $r(G) = |V| - c$ where c is number of connected components
- **Nullity:** $\mu(G) = |E| - r(G) =$ number of independent cycles

$$|V| = 4, |E| = 6, c = 1$$

Original Graph

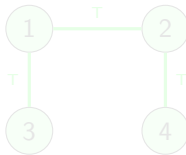


$$r(G) = |V| - c = 4 - 1 = 3$$

(tree edges)

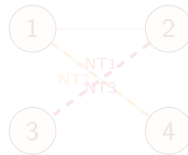
$$|E'| = 3 = r(G)$$

Spanning Tree



$$\mu(G) = |E| - r(G) = 3$$

Non-Tree Edges



$$\mu(G) = |E| - r(G) = 6 - 3 = 3$$

(non-tree edges)



Rank and Nullity

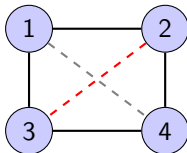
Rank and Nullity

For a graph $G = (V, E)$:

- **Rank:** $r(G) = |V| - c$ where c is number of connected components
- **Nullity:** $\mu(G) = |E| - r(G) =$ number of independent cycles

$$|V| = 4, |E| = 6, c = 1$$

Original Graph

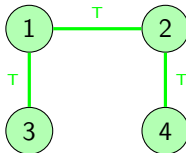


$$r(G) = |V| - c = 4 - 1 = 3$$

(tree edges)

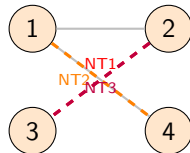
$$|E'| = 3 = r(G)$$

Spanning Tree



$$\mu(G) = |E| - r(G) = 3$$

Non-Tree Edges



$$\mu(G) = |E| - r(G)$$

$$= 6 - 3 = 3$$

(non-tree edges)



Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary



Graph Traversal: Overview

Motivation

Traversal algorithms systematically visit all vertices of a graph. Essential for:

- Finding connected components
- Detecting cycles
- Computing shortest paths
- Topological sorting



Graph Traversal: Overview

Two Primary Strategies

- **Breadth-First Search (BFS):** Explores layer-by-layer; uses queue
- **Depth-First Search (DFS):** Explores recursively down branches; uses stack

Distinction from Trees

Trees are special graphs; traversal on trees is simpler (acyclic). Same algorithms work on general graphs with cycle handling.



Graph Traversal: Overview

Two Primary Strategies

- **Breadth-First Search (BFS):** Explores layer-by-layer; uses queue
- **Depth-First Search (DFS):** Explores recursively down branches; uses stack

Distinction from Trees

Trees are special graphs; traversal on trees is simpler (acyclic). Same algorithms work on general graphs with cycle handling.



Breadth-First Search (BFS)

Algorithm Definition

BFS explores vertices in **level order**. Starting from a source vertex, visit all neighbors, then their neighbors, etc.

Data Structure: Queue (FIFO)

Pseudocode

- 1 Mark source as visited, enqueue it
- 2 While queue not empty:
 - 1 Dequeue vertex u
 - 2 For each unvisited neighbor v of u :
 - 1 Mark v visited, enqueue v , record u as parent of v

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ queue storage in worst case

Breadth-First Search (BFS)

Algorithm Definition

BFS explores vertices in **level order**. Starting from a source vertex, visit all neighbors, then their neighbors, etc.

Data Structure: Queue (FIFO)

Pseudocode

- ➊ Mark source as visited, enqueue it
- ➋ While queue not empty:
 - ➊ Dequeue vertex u
 - ➋ For each unvisited neighbor v of u :
 - ➊ Mark v visited, enqueue v , record u as parent of v

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ queue storage in worst case

Breadth-First Search (BFS)

Algorithm Definition

BFS explores vertices in **level order**. Starting from a source vertex, visit all neighbors, then their neighbors, etc.

Data Structure: Queue (FIFO)

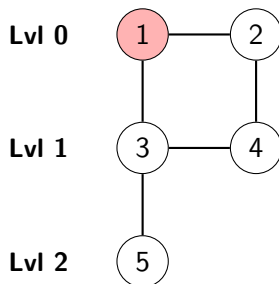
Pseudocode

- ➊ Mark source as visited, enqueue it
- ➋ While queue not empty:
 - ➊ Dequeue vertex u
 - ➋ For each unvisited neighbor v of u :
 - ➊ Mark v visited, enqueue v , record u as parent of v

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ queue storage in worst case

BFS Visual Example



Queue (FIFO)

Start: [1]

Step 1: [2, 3]

Step 2: [3, 4]

Step 3: [4, 5]

Traversal Order

BFS from node 1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Visits all vertices at distance d before distance $d + 1$.



Depth-First Search (DFS)

Algorithm Definition

DFS explores vertices by **going deep** into the graph. Recursively visit unvisited neighbors before backtracking.

Data Structure: Stack (LIFO) or recursion

Pseudocode (Recursive)

- 1 Mark u as visited
- 2 For each unvisited neighbor v of u :
 - 1 Record u as parent of v
 - 2 Recursively call $\text{DFS}(v)$

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ recursion stack in worst case

Depth-First Search (DFS)

Algorithm Definition

DFS explores vertices by **going deep** into the graph. Recursively visit unvisited neighbors before backtracking.

Data Structure: Stack (LIFO) or recursion

Pseudocode (Recursive)

- ➊ Mark u as visited
- ➋ For each unvisited neighbor v of u :
 - ➊ Record u as parent of v
 - ➋ Recursively call $\text{DFS}(v)$

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ recursion stack in worst case

Depth-First Search (DFS)

Algorithm Definition

DFS explores vertices by **going deep** into the graph. Recursively visit unvisited neighbors before backtracking.

Data Structure: Stack (LIFO) or recursion

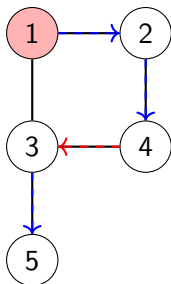
Pseudocode (Recursive)

- ➊ Mark u as visited
- ➋ For each unvisited neighbor v of u :
 - ➊ Record u as parent of v
 - ➋ Recursively call $\text{DFS}(v)$

Complexity

- **Time:** $O(|V| + |E|)$ each vertex/edge visited once
- **Space:** $O(|V|)$ recursion stack in worst case

DFS Visual Example



Stack (LIFO)

Start: [1]

Step 1: [2]

Step 2: [4]

Step 3: [3, 5]

Traversal Order

DFS from node 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 5$

Goes as deep as possible before backtracking.



BFS vs. DFS Comparison

Comparison of Traversal Algorithms

Aspect	BFS	DFS
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (queue)	$O(V)$ (stack)
Data Structure	Queue	Stack/Recursion
Visits by	Levels	Depth first
Shortest path	✓ (unweighted)	✗
Memory efficient	Variable	Variable

Use Case Guidelines

- **BFS:** Shortest path (unweighted), level-based exploration, checking connectivity
- **DFS:** Cycle detection, topological sorting, connected components, backtracking problems



BFS vs. DFS Comparison

Comparison of Traversal Algorithms

Aspect	BFS	DFS
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (queue)	$O(V)$ (stack)
Data Structure	Queue	Stack/Recursion
Visits by	Levels	Depth first
Shortest path	✓ (unweighted)	✗
Memory efficient	Variable	Variable

Use Case Guidelines

- **BFS:** Shortest path (unweighted), level-based exploration, checking connectivity
- **DFS:** Cycle detection, topological sorting, connected components, backtracking problems



Applications of Tree Traversal

BFS Applications

- Shortest path (unweighted)
- Level order traversal
- Connected components
- Bipartiteness check
- Social network *degrees*

DFS Applications

- Cycle detection
- Topological sort
- Strongly connected components
- Articulation points
- Backtracking: puzzles, mazes

Note

Both algorithms have identical time and space complexity but different *traversal orders* make them suitable for different problems.



Applications of Tree Traversal

BFS Applications

- Shortest path (unweighted)
- Level order traversal
- Connected components
- Bipartiteness check
- Social network *degrees*

DFS Applications

- Cycle detection
- Topological sort
- Strongly connected components
- Articulation points
- Backtracking: puzzles, mazes

Note

Both algorithms have identical time and space complexity but different *traversal orders* make them suitable for different problems.



Outline

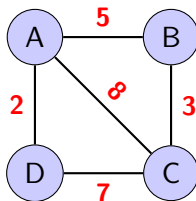
- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs**
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary



Introduction to Weighted Graphs

Definition

A **weighted graph** assigns a numerical **weight** $w(e)$ to each edge e , representing cost, distance, capacity, or other attributes.



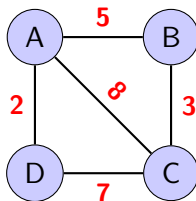
Weight Interpretation

- **Distance/Cost:** Edge weights represent real-world costs
- **Total Weight:** $w(G') = \sum_{e \in E'} w(e)$
- **Optimization Goal:** Find subgraph (path, tree, etc.) with minimum total weight

Introduction to Weighted Graphs

Definition

A **weighted graph** assigns a numerical **weight** $w(e)$ to each edge e , representing cost, distance, capacity, or other attributes.



Weight Interpretation

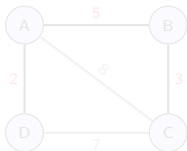
- **Distance/Cost:** Edge weights represent real-world costs
- **Total Weight:** $w(G') = \sum_{e \in E'} w(e)$
- **Optimization Goal:** Find subgraph (path, tree, etc.) with minimum total weight

Minimum Spanning Tree (MST)

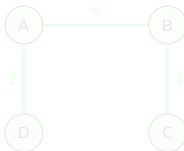
Definition

A **Minimum Spanning Tree** of a weighted connected graph is a spanning tree with minimum total weight.

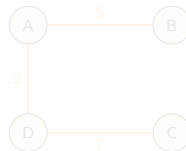
Graph (weight=25)



MST 1 (weight=10)



MST 2 (weight=12)



MST Properties

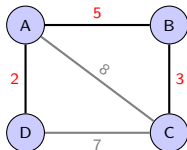
- Has **minimum total weight** among all spanning trees
- Always has $|V| - 1$ edges
- May not be unique** if multiple edge weights are equal

Minimum Spanning Tree (MST)

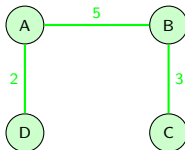
Definition

A **Minimum Spanning Tree** of a weighted connected graph is a spanning tree with minimum total weight.

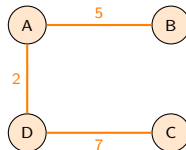
Graph (weight=25)



MST 1 (weight=10)



MST 2 (weight=12)



MST Properties

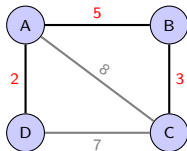
- Has **minimum total weight** among all spanning trees
- Always has $|V| - 1$ edges
- May not be unique if multiple edge weights are equal

Minimum Spanning Tree (MST)

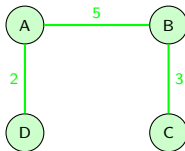
Definition

A **Minimum Spanning Tree** of a weighted connected graph is a spanning tree with minimum total weight.

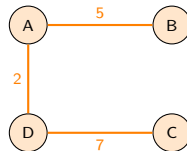
Graph (weight=25)



MST 1 (weight=10)



MST 2 (weight=12)



MST Properties

- Has **minimum total weight** among all spanning trees
- Always has $|V| - 1$ edges
- **May not be unique** if multiple edge weights are equal

Outline

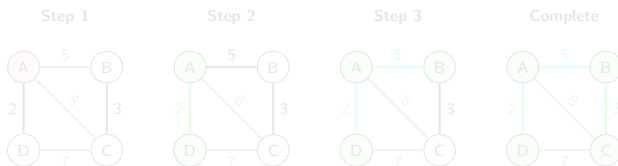
- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms**
- 5 Analysis
- 6 Summary



Prim's Algorithm

Algorithm Overview

Greedy approach: Start with a vertex and grow MST by adding minimum-weight edge connecting tree to non-tree vertices.



Complexity Analysis

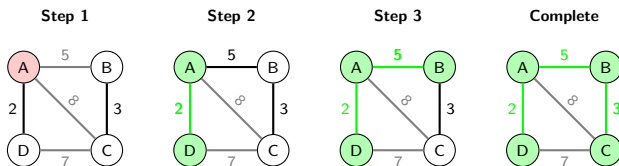
- Naive: $O(|V|^2)$ | With binary heap: $O((|V| + |E|) \log |V|)$



Prim's Algorithm

Algorithm Overview

Greedy approach: Start with a vertex and grow MST by adding minimum-weight edge connecting tree to non-tree vertices.



Complexity Analysis

- **Naive:** $O(|V|^2)$ | **With binary heap:** $O((|V| + |E|) \log |V|)$



Kruskal's Algorithm

Algorithm Overview

Greedy approach: Sort edges by weight, add edges in order if they don't create a cycle (using Union-Find).

Sorted Edges: A-D(2), B-C(3), A-B(5), D-C(7)

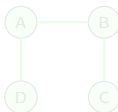
1. Add A-D



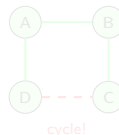
2. Add B-C



3. Add A-B



4. Skip D-C



Complexity Analysis

- **Total:** $O(|E| \log |E|)$ dominated by sorting + union-find operations



Kruskal's Algorithm

Algorithm Overview

Greedy approach: Sort edges by weight, add edges in order if they don't create a cycle (using Union-Find).

Sorted Edges: A-D(2), B-C(3), A-B(5), D-C(7)

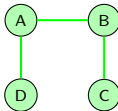
1. Add A-D



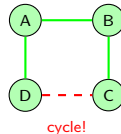
2. Add B-C



3. Add A-B



4. Skip D-C



Complexity Analysis

- **Total:** $O(|E| \log |E|)$ dominated by sorting + union-find operations



Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis**
- 6 Summary



Correctness of MST Algorithms

Cut Property

For any cut $(S, V \setminus S)$ of the graph, the minimum-weight edge crossing the cut must be in some MST.

Proof Strategy

- **Prim:** At each step, minimum-weight edge from tree to non-tree satisfies cut property
- **Kruskal:** Adding minimum-weight edge that doesn't create cycle respects cut property for its cut

Corollary

Both Prim's and Kruskal's algorithms produce optimal solutions. If all edge weights are distinct, MST is unique.



Correctness of MST Algorithms

Cut Property

For any cut $(S, V \setminus S)$ of the graph, the minimum-weight edge crossing the cut must be in some MST.

Proof Strategy

- **Prim:** At each step, minimum-weight edge from tree to non-tree satisfies cut property
- **Kruskal:** Adding minimum-weight edge that doesn't create cycle respects cut property for its cut

Corollary

Both Prim's and Kruskal's algorithms produce optimal solutions. If all edge weights are distinct, MST is unique.



Correctness of MST Algorithms

Cut Property

For any cut $(S, V \setminus S)$ of the graph, the minimum-weight edge crossing the cut must be in some MST.

Proof Strategy

- **Prim:** At each step, minimum-weight edge from tree to non-tree satisfies cut property
- **Kruskal:** Adding minimum-weight edge that doesn't create cycle respects cut property for its cut

Corollary

Both Prim's and Kruskal's algorithms produce optimal solutions. If all edge weights are distinct, MST is unique.



Algorithm Comparison

Prim vs. Kruskal Comparison

Aspect	Prim's	Kruskal's
Time Complexity	$O(V ^2)$ or $O((E + V) \log V)$	$O(E \log E)$
Space Complexity	$O(V)$ or $O(V + E)$	$O(V + E)$
Best for	Dense graphs	Sparse graphs
Data Structure	Priority queue	Sorting + Union-Find
Parallel	Difficult	More suitable

Selection Guideline

- $|E| \approx |V|^2$: Use Prim's (dense)
- $|E| \ll |V|^2$: Use Kruskal's (sparse)
- **Practical choice:** Kruskal's more commonly implemented due to simplicity



Algorithm Comparison

Prim vs. Kruskal Comparison

Aspect	Prim's	Kruskal's
Time Complexity	$O(V ^2)$ or $O((E + V) \log V)$	$O(E \log E)$
Space Complexity	$O(V)$ or $O(V + E)$	$O(V + E)$
Best for	Dense graphs	Sparse graphs
Data Structure	Priority queue	Sorting + Union-Find
Parallel	Difficult	More suitable

Selection Guideline

- $|E| \approx |V|^2$: Use Prim's (dense)
- $|E| \ll |V|^2$: Use Kruskal's (sparse)
- **Practical choice:** Kruskal's more commonly implemented due to simplicity



Complexity Considerations

Practical Factors

- **Graph density:** Determines dominance of edge vs. vertex operations
- **Data structures:** Choice of implementation (array, heap, union-find) affects constants
- **Edge weight distribution:** Affects number of comparisons in practice
- **Memory access patterns:** Cache locality impacts real-world performance



Complexity Considerations

Advanced Techniques

- **Borůvka's algorithm:** Parallel-friendly MST with $O(|E| \log |V|)$ complexity
- **Randomized approaches:** Expected linear time under certain conditions
- **Approximation methods:** For very large graphs requiring approximate MSTs



Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary**



Program Topics Covered

✓ Spanning Trees

Definition and properties

✓ Rank and Nullity

Graph cycle structure

✓ Fundamental Circuits

Cycle generation from spanning trees

✓ Tree Traversal

BFS and DFS algorithms

✓ Weighted Graphs

Edge weights and costs

✓ MST Algorithms

Prim's and Kruskal's



Outline

- 1 Spanning Trees
- 2 Tree Traversal and Search Algorithms
- 3 Weighted Graphs
- 4 Minimum Spanning Tree Algorithms
- 5 Analysis
- 6 Summary



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

